

Week 7 Lecture Plan

Futures and Async Design: Pipelines, Failure, Timeout, and Fan-Out/Fan-In

Instructor: Dr. Mohamad Aoude

Pre-Class Preparation

- ☐ Week 7 lab compiles: `week7-lab/week7-day1-async-testing-antipatterns`
- ☐ Smoke test passes: `gradlew.bat test -tests "edu.lu.concurrency.week7.day1.DemoSmokeTest"`
- ☐ Lecture PDFs ready: `Lecture_07_01_nar.pdf` through `Lecture_07_04_nar.pdf`
- ☐ Lab manual ready: `Lab_Week07_AsyncTestingAntipatterns.pdf`
- ☐ Terminal ready for demos: pipeline, timeout/fallback, reproduction strategy, stress classification, fan-out/fan-in, NIO counterpoint
- ☐ Keep `ASYNC_PIPELINE_SEQUENCE.md`, `ANTI_PATTERNS.md`, and `CONCURRENCY_SCORECARD.md` visible

Learning Targets

By the end of Week 7, students must be able to:

1. Build a small `CompletableFuture` pipeline using `supplyAsync`, transformation, and explicit completion.
2. Explain why async design is about composition and waiting, not simply more threads.
3. Add timeout and fallback behavior to slow service calls.
4. Convert async failures into explicit fallback results without silently hiding errors.
5. Implement fan-out/fan-in using `CompletableFuture.allOf` and explicit aggregation.
6. Describe how stress-style tests classify acceptable, interesting, and forbidden outcomes.
7. Compare blocking thread-per-request design with NIO-style event multiplexing.

Lecture Structure

Segment	Material	Goal
Part 1	<code>Lecture_07_01_nar</code>	<code>CompletableFuture</code> , async pipeline shape, executor choice
Part 2	<code>Lecture_07_02_nar</code>	Timeout, fallback, exception handling, anti-patterns
Part 3	<code>Lecture_07_03_nar</code>	Fan-out/fan-in, explicit aggregation, sequence diagram
Part 4	<code>Lecture_07_04_nar</code>	Testing traps, stress-style thinking, NIO counterpoint, Week 8 bridge

Recommended Time Budget

Time	Activity	Notes
0–10 min	Week 6 handoff	Fork/Join splits CPU work; Week 7 composes async service work.
10–35 min	Part 1 slides and Demo01	Build and read a basic future pipeline.
35–60 min	Part 2 slides and Demo02	Add timeout/fallback and discuss failure semantics.
60–90 min	Part 3 slides and Demo06	Fan out service calls, wait for all, aggregate explicitly.
90–110 min	Part 4 slides and Demo03–Demo05	Why naive tests miss races; stress-style classification.
110–120 min	Demo07–Demo08 and exit ticket	NIO counterpoint and anti-pattern check.

Live Demo Checklist

- ☐ **Demo01:** Run the basic `CompletableFuture` pipeline.
- ☐ **Demo02:** Compare fast service completion with timeout fallback.
- ☐ **Demo03:** Show a race that appears harmless until amplified.
- ☐ **Demo04:** Translate a flaky observation into a reproduction strategy.
- ☐ **Demo05:** Classify outcomes as acceptable, interesting, or forbidden.
- ☐ **Demo06:** Walk through fan-out/fan-in and identify fallback paths.
- ☐ **Demo07:** Compare blocking-per-connection with selector-style multiplexing.
- ☐ **Demo08:** Close with the anti-pattern list.

Student Work Contract

Students implement only:

- `Ex1_AsyncTimeoutFallback`
- `Ex2_ReproductionStrategy`
- `Ex3_StressCounterCheck`
- `Ex4_FanOutFanInPipeline`
- `Ex5_NioCounterpoint`

They must not edit tests. They pass the lab when:

- `gradlew.bat studentCheck` passes,
- the completed scorecard explains timeout and fallback behavior,
- the fan-out/fan-in sequence diagram identifies the slow and failing service paths.

Exit Ticket

Ask students to answer in three sentences:

1. What is the difference between async composition and CPU parallelism?
2. Where should timeout and fallback logic appear in a service pipeline?
3. Why is swallowing async exceptions without recording fallback dangerous?